



COMP30026

Models of Computation

Cezary Kaliszyk and William Umboh

Lecture 9

Predicate Logic: Unification and Resolution



Predicate Logic Extends Propositional L.

How?



Predicate Logic Extends Propositional L.

How?

- Syntax?
- Semantics?
- Proofs?



Predicate Logic Extends Propositional L.

How?

- Syntax?
- Semantics?
 - Universe, Interpretation, ...?
- Proofs?
 - Resolution



Notation for Variables and Constants

Recall our convention:

Constants

Letters from the start of the alphabet ($a, b, c, \dots$)

Variables

Letters from the end of the alphabet ($u, v, x, y, \dots$)

This distinction is very important in what follows.

Substitutions

Definition: substitution

- function from variables to terms
- denoted: $\theta = \{x_1 \mapsto t_1, x_2 \mapsto t_2, \dots, x_n \mapsto t_n\}$

Substitutions

Definition: substitution

- function from variables to terms
- denoted: $\theta = \{x_1 \mapsto t_1, x_2 \mapsto t_2, \dots, x_n \mapsto t_n\}$

Can be applied to a formula or term E

- **simultaneously** replaces all occurrence of x_i in E by t_i
- denoted by $E[\theta]$

Substitutions

Definition: substitution

- function from variables to terms
- denoted: $\theta = \{x_1 \mapsto t_1, x_2 \mapsto t_2, \dots, x_n \mapsto t_n\}$

Can be applied to a formula or term E

- **simultaneously** replaces all occurrence of x_i in E by t_i
- denoted by $E[\theta]$

Example

If E is $P(f(x), g(y, y, b))$, and

$$\theta = \{x \mapsto h(u), y \mapsto a, z \mapsto c\}$$

then $E[\theta]$ is $P(f(h(u)), g(a, a, b))$.

Definition: **unifier** of two expressions s and t
a substitution θ such that $s[\theta] = t[\theta]$

s and t are **unifiable** iff there exists a unifier for s and t .

Definition: **unifier** of two expressions s and t
a substitution θ such that $s[\theta] = t[\theta]$

s and t are **unifiable** iff there exists a unifier for s and t .

Example

$L(x)$ and $L(c)$ are unifiable:

Definition: **unifier** of two expressions s and t
a substitution θ such that $s[\theta] = t[\theta]$

s and t are **unifiable** iff there exists a unifier for s and t .

Example

$L(x)$ and $L(c)$ are unifiable:

$$\theta = \{x \mapsto c\}$$

Both $L(x)[\theta]$ and $L(c)[\theta]$ are $L(c)$.

Definition: **unifier** of two expressions s and t
a substitution θ such that $s[\theta] = t[\theta]$

s and t are **unifiable** iff there exists a unifier for s and t .

Example

$L(x)$ and $L(c)$ are unifiable:

$$\theta = \{x \mapsto c\}$$

Both $L(x)[\theta]$ and $L(c)[\theta]$ are $L(c)$.

Question: Are $L(a)$ and $L(c)$ unifiable?



Most General Unifiers

Question: Are $L(x)$ and $L(y)$ unifiable?

Most General Unifiers

Question: Are $L(x)$ and $L(y)$ unifiable?

Definition: most general unifier (mgu)

substitution θ for s and t such that

1. θ is a unifier for s and t , and
2. every other unifier σ of s and t can be expressed as $\tau \circ \theta$ for some substitution τ .

(The composition $\tau \circ \theta$ is the substitution which first applies θ , and then applies τ to that result.)

Claim: If s and t are unifiable, they have a mgu.



Unifier Examples

1. $P(x, a)$ and $P(b, c)$
2. $P(f(x), y)$ and $P(a, w)$
3. $P(x, c)$ and $P(a, y)$
4. $P(f(x), c)$ and $P(f(a), y)$
5. $P(x)$ and $P(f(x))$

Unifier Examples

1. $P(x, a)$ and $P(b, c)$ are not unifiable.
2. $P(f(x), y)$ and $P(a, w)$ are not unifiable.
3. $P(x, c)$ and $P(a, y)$ are unifiable using $\{x \mapsto a, y \mapsto c\}$.
4. $P(f(x), c)$ and $P(f(a), y)$ also unifiable using $\{x \mapsto a, y \mapsto c\}$.
5. $P(x)$ and $P(f(x))$ are **not** unifiable.

If we were allowed to have a substitution $\{x \mapsto f(f(f(\dots)))\}$, that would be a unifier for the last example. But we cannot have that, as terms must be **finite**.

More Unifier Examples

Now consider $P(f(x), g(y, a))$ and $P(f(a), g(z, a))$.

The following are all unifiers, so which is “best”?

- $A = \{x \mapsto a, y \mapsto z\}$
- $B = \{x \mapsto a, y \mapsto a, z \mapsto a\}$
- $C = \{x \mapsto a, y \mapsto g(b, f(u)), z \mapsto g(b, f(u))\}$
- $D = \{x \mapsto a, z \mapsto y\}$

More Unifier Examples

Now consider $P(f(x), g(y, a))$ and $P(f(a), g(z, a))$.

The following are all unifiers, so which is “best”?

- $A = \{x \mapsto a, y \mapsto z\}$
- $B = \{x \mapsto a, y \mapsto a, z \mapsto a\}$
- $C = \{x \mapsto a, y \mapsto g(b, f(u)), z \mapsto g(b, f(u))\}$
- $D = \{x \mapsto a, z \mapsto y\}$

A and D are **mgus**. They avoid making unnecessary commitments.

B needlessly commits y and z to be a .

Note that $B = \{y \mapsto a\} \circ D$.



A Syntactic Unification Algorithm

Input: Two expressions s and t .

Output: If they are unifiable: a most general unifier for s and t ; otherwise 'failure'.

Algorithm:

1. Start with this set consisting of one equation: $\{s = t\}$.
2. As long as some equation in the set has one of the six forms listed on the next slide, perform the corresponding action.
3. Return the result.



Unification: Solving Term Equations

In the following, let x be a variable and let F and G be function or predicate symbols.

1. $F(s_1, \dots, s_n) = F(t_1, \dots, t_n)$:
 - Replace by the n equations $s_1 = t_1, \dots, s_n = t_n$.



Unification: Solving Term Equations

In the following, let x be a variable and let F and G be function or predicate symbols.

1. $F(s_1, \dots, s_n) = F(t_1, \dots, t_n)$:
 - Replace by the n equations $s_1 = t_1, \dots, s_n = t_n$.
2. $F(s_1, \dots, s_n) = G(t_1, \dots, t_m)$ where $F \neq G$ or $n \neq m$:
 - Halt, returning 'failure'.



Unification: Solving Term Equations

In the following, let x be a variable and let F and G be function or predicate symbols.

1. $F(s_1, \dots, s_n) = F(t_1, \dots, t_n)$:
 - Replace by the n equations $s_1 = t_1, \dots, s_n = t_n$.
2. $F(s_1, \dots, s_n) = G(t_1, \dots, t_m)$ where $F \neq G$ or $n \neq m$:
 - Halt, returning 'failure'.
3. $x = x$:
 - Delete the equation.



Unification: Solving Term Equations

In the following, let x be a variable and let F and G be function or predicate symbols.

1. $F(s_1, \dots, s_n) = F(t_1, \dots, t_n)$:
 - Replace by the n equations $s_1 = t_1, \dots, s_n = t_n$.
2. $F(s_1, \dots, s_n) = G(t_1, \dots, t_m)$ where $F \neq G$ or $n \neq m$:
 - Halt, returning 'failure'.
3. $x = x$:
 - Delete the equation.
4. $t = x$ where t is not a variable:
 - Replace the equation by $x = t$.



Unification: Solving Term Equations

In the following, let x be a variable and let F and G be function or predicate symbols.

1. $F(s_1, \dots, s_n) = F(t_1, \dots, t_n)$:
 - Replace by the n equations $s_1 = t_1, \dots, s_n = t_n$.
2. $F(s_1, \dots, s_n) = G(t_1, \dots, t_m)$ where $F \neq G$ or $n \neq m$:
 - Halt, returning 'failure'.
3. $x = x$:
 - Delete the equation.
4. $t = x$ where t is not a variable:
 - Replace the equation by $x = t$.
5. $x = t$ where t is not x but x occurs in t :
 - Halt, returning 'failure'.



Unification: Solving Term Equations

In the following, let x be a variable and let F and G be function or predicate symbols.

1. $F(s_1, \dots, s_n) = F(t_1, \dots, t_n)$:
 - Replace by the n equations $s_1 = t_1, \dots, s_n = t_n$.
2. $F(s_1, \dots, s_n) = G(t_1, \dots, t_m)$ where $F \neq G$ or $n \neq m$:
 - Halt, returning 'failure'.
3. $x = x$:
 - Delete the equation.
4. $t = x$ where t is not a variable:
 - Replace the equation by $x = t$.
5. $x = t$ where t is not x but x occurs in t :
 - Halt, returning 'failure'.
6. $x = t$ where t contains no x :
 - Save. Replace x by t in other equations.



Solving Term Equations: Example 1

Starting from

$$f(h(y), g(y, a), z) = f(x, g(v, v), b)$$

Solving Term Equations: Example 1

Starting from

$$f(h(y), g(y, a), z) = f(x, g(v, v), b)$$

we rewrite:

$$\begin{array}{rcl} & x & = h(y) \\ \stackrel{(1,4)}{\implies} & g(y, a) & = g(v, v) \\ & z & = b \end{array}$$

Solving Term Equations: Example 1

Starting from

$$f(h(y), g(y, a), z) = f(x, g(v, v), b)$$

we rewrite:

$$\begin{array}{lcl} & x & = h(y) \\ \stackrel{(1,4)}{\implies} & g(y, a) & = g(v, v) \\ & z & = b \end{array} \quad \stackrel{(1,4)}{\implies} \quad \begin{array}{lcl} x & = & h(y) \\ y & = & v \\ v & = & a \\ z & = & b \end{array}$$

Solving Term Equations: Example 1

Starting from

$$f(h(y), g(y, a), z) = f(x, g(v, v), b)$$

we rewrite:

$$\begin{array}{lcl}
 \begin{array}{l}
 \xRightarrow{(1,4)} \begin{array}{l} x = h(y) \\ g(y, a) = g(v, v) \\ z = b \end{array} \\
 \end{array}
 & \xRightarrow{(1,4)} & \begin{array}{l} x = h(y) \\ y = v \\ v = a \\ z = b \end{array}
 \end{array}
 \quad
 \begin{array}{l}
 \begin{array}{l} x = h(a) \\ y = a \\ v = a \\ z = b \end{array} \\
 \end{array}
 \xRightarrow{(6,6)}$$

Solving Term Equations: Example 1

Starting from

$$f(h(y), g(y, a), z) = f(x, g(v, v), b)$$

we rewrite:

$$\begin{array}{lcl} \begin{array}{l} x = h(y) \\ g(y, a) = g(v, v) \\ z = b \end{array} & \xRightarrow{(1,4)} & \begin{array}{l} x = h(y) \\ y = v \\ v = a \\ z = b \end{array} & \xRightarrow{(6,6)} & \begin{array}{l} x = h(a) \\ y = a \\ v = a \\ z = b \end{array} \end{array}$$

The last set is in **normal form** and corresponds to the substitution

$$\{x \mapsto h(a), y \mapsto a, v \mapsto a, z \mapsto b\}$$

which indeed unifies the two original terms.

Solving Term Equations: Example 2

Starting from

$$f(x, a, x) = f(h(z, b), y, h(z, y))$$

we rewrite:

$$\begin{array}{ccc} \xRightarrow{(1,4)} & \begin{array}{l} x = h(z, b) \\ y = a \\ x = h(z, y) \end{array} & \xRightarrow{(6)} \begin{array}{l} x = h(z, b) \\ y = a \\ h(z, b) = h(z, y) \end{array} \xRightarrow{(1,4)} \begin{array}{l} x = h(z, b) \\ y = a \\ z = z \\ y = b \end{array} \end{array}$$

$$\begin{array}{ccc} \xRightarrow{(3)} & \begin{array}{l} x = h(z, b) \\ y = a \\ y = b \end{array} & \xRightarrow{(6)} \begin{array}{l} x = h(z, b) \\ y = a \\ a = b \end{array} \xRightarrow{(2)} \text{failure} \end{array}$$

So the two original terms are not unifiable.

Solving Term Equations: Example 3

Starting from

$$f(x, g(v, v), x) = f(h(y), g(y, z), z)$$

we rewrite:

$$\begin{array}{ccc} x & = & h(y) \\ \xRightarrow{(1)} g(v, v) & = & g(y, z) \xRightarrow{(6,4)} g(v, v) & = & g(y, z) \\ x & = & z & & z & = & h(y) \end{array}$$

$$\begin{array}{ccccc} x & = & h(y) & & x & = & h(y) & & x & = & h(y) \\ \xRightarrow{(1)} v & = & y & \xRightarrow{(6)} z & = & y & \xRightarrow{(6)} z & = & y & \xRightarrow{(5)} \text{fail} \\ v & = & z & & v & = & z & & v & = & y \\ z & = & h(y) & & z & = & h(y) & & y & = & h(y) \end{array}$$

This is “failure by occurs check”: The algorithm fails as soon as we discover the equation $y = h(y)$.

Term Equations as Substitutions

This algorithm always halts.

If the result is 'failure', no unifier exists.

Otherwise, the term equation system is in **normal form**:

- Every LHS is a different variable.
- No LHS appears in any RHS.

If the normal form is $\{x_1 = t_1, \dots, x_n = t_n\}$ then

$$\{x_1 \mapsto t_1, \dots, x_n \mapsto t_n\}$$

is a mgu for the input terms.

Resolution for Predicate Logic

Let P_1 and P_2 be **unifiable** atomic formulas with **no variables in common**. Let θ be the unifier.

Let C_1 and C_2 be disjunctive clauses.

$$\frac{C_1 \cup \{P_1\} \quad C_2 \cup \{\neg P_2\}}{(C_1 \cup C_2)[\theta]}$$



Automated Inference with Predicate Logic

- Every shark eats a tadpole

$$\forall x(S(x) \rightarrow \exists y(T(y) \wedge E(x, y)))$$

- All large white fish are sharks

$$\forall x(W(x) \rightarrow S(x))$$

- Camilla is a large white fish living in deep water

$$W(camilla) \wedge D(camilla)$$

- Any tadpole eaten by a deep water fish is miserable

$$\forall z((T(z) \wedge \exists y(D(y) \wedge E(y, z))) \rightarrow M(z))$$

- Therefore some tadpole is miserable

$$\therefore \exists z(T(z) \wedge M(z))$$



Tadpoles in Clausal Form

- Every shark eats a tadpole

$$\{\neg S(x), T(f(x))\}, \{\neg S(x), E(x, f(x))\}$$

- All large white fish are sharks

$$\{\neg W(x), S(x)\}$$

- Camilla is a large white fish living in deep water

$$\{W(camilla)\}, \{D(camilla)\}$$

- Any tadpole eaten by a deep water fish is miserable

$$\{\neg T(z), \neg D(y), \neg E(y, z), M(z)\}$$

- Negation of: Some tadpole is miserable.

$$\{\neg T(z), \neg M(z)\}$$

A Refutation

In the presentation we need to save space

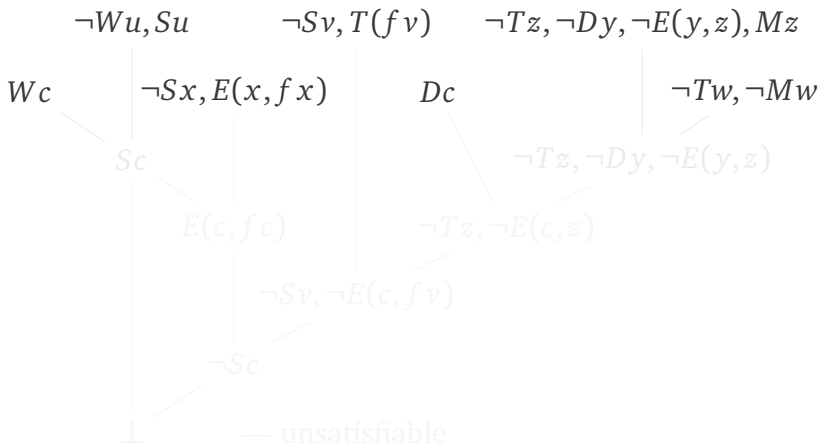
For example, we write $\neg Wu, Su$ for clause $\{\neg W(u), S(u)\}$

Do not do this in assignments / exam (list form)!

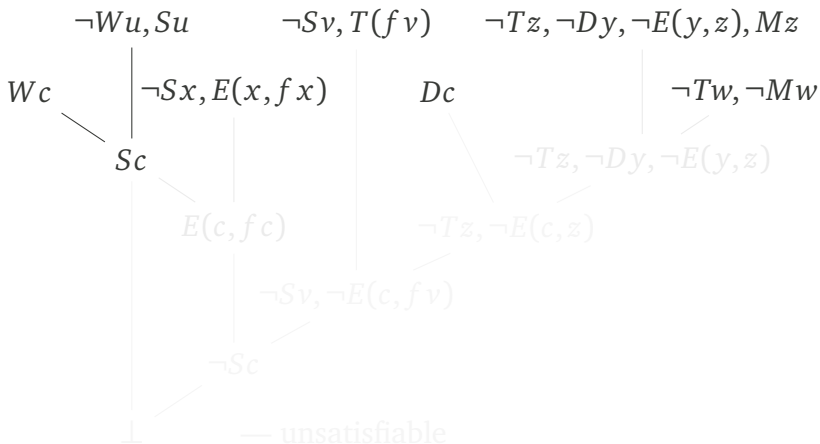
$\neg Wu, Su$ $\neg Sv, T(fv)$ $\neg Tz, \neg Dy, \neg E(y, z), Mz$

Wc $\neg Sx, E(x, fx)$ Dc $\neg Tw, \neg Mw$

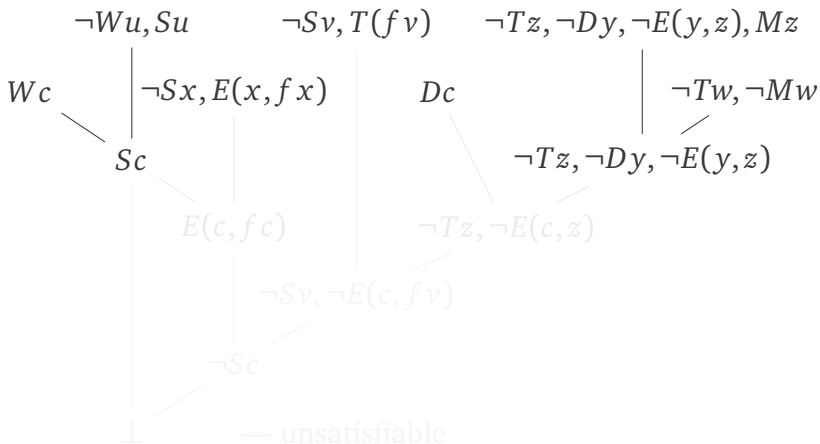
A Refutation for the Tadpole Example



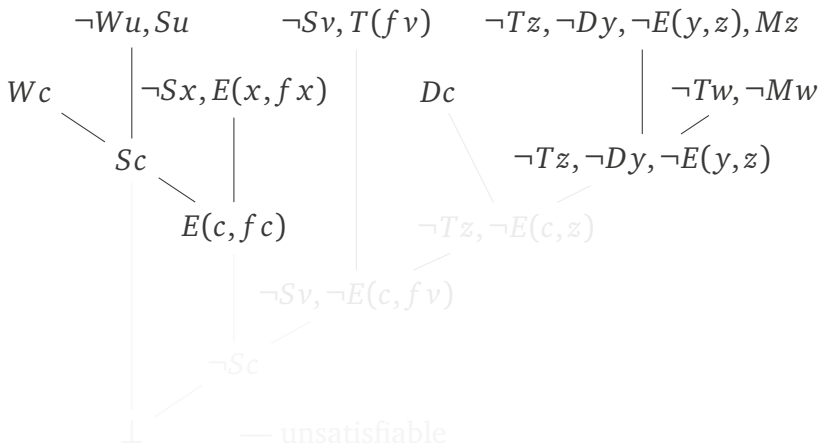
A Refutation for the Tadpole Example



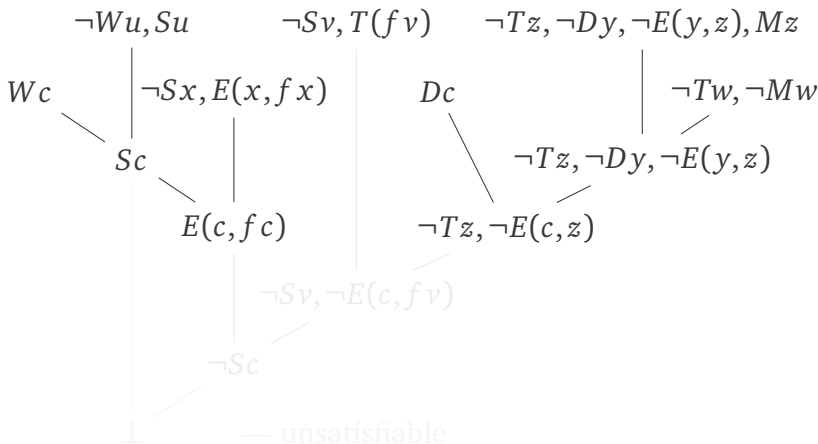
A Refutation for the Tadpole Example



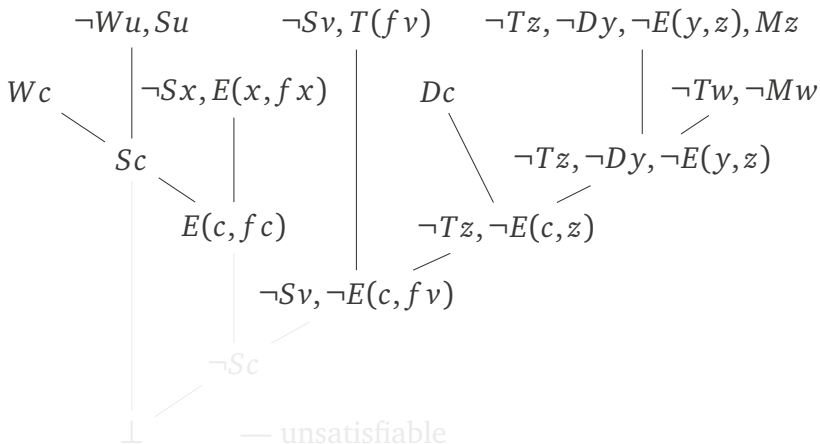
A Refutation for the Tadpole Example



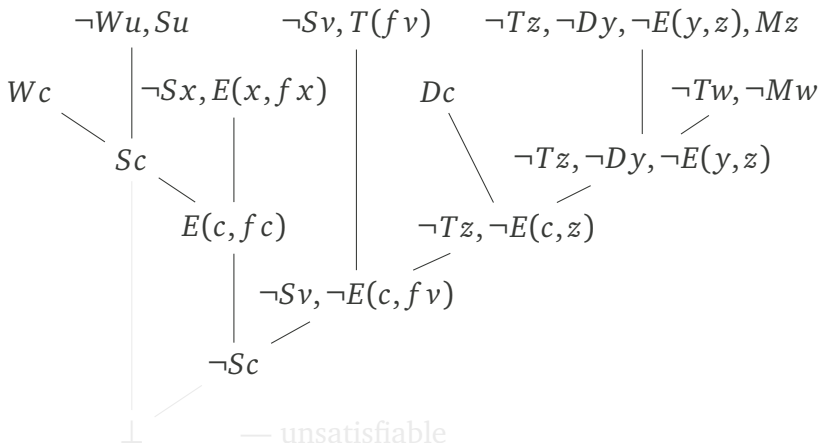
A Refutation for the Tadpole Example



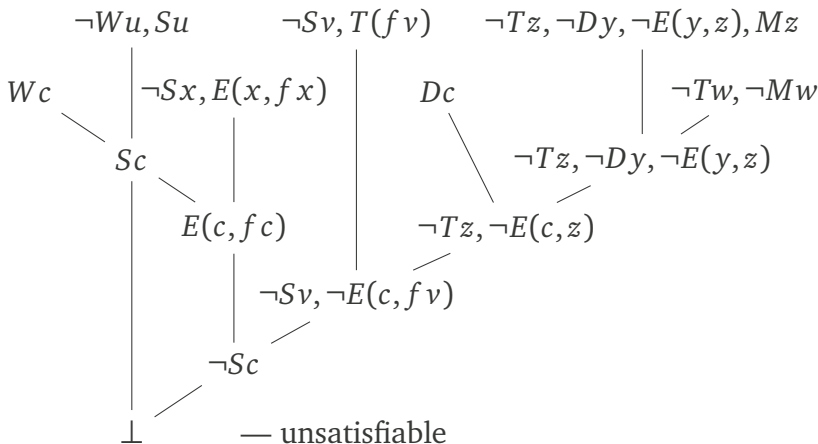
A Refutation for the Tadpole Example



A Refutation for the Tadpole Example



A Refutation for the Tadpole Example





Resolution Exercise

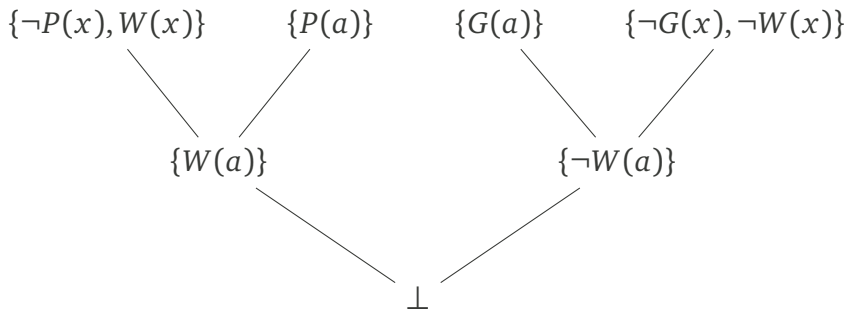
Using resolution, justify this argument:

- All philosophers are wise $\forall x(P(x) \rightarrow W(x))$
- Some Greeks are philosophers $\exists x(G(x) \wedge P(x))$
- Therefore some Greeks are wise $\exists x(G(x) \wedge W(x))$

Resolution Exercise

Using resolution, justify this argument:

- All philosophers are wise $\forall x(P(x) \rightarrow W(x))$
- Some Greeks are philosophers $\exists x(G(x) \wedge P(x))$
- Therefore some Greeks are wise $\exists x(G(x) \wedge W(x))$



Factoring

Second kind of inference: **factoring**.

Let C be a clause and let $A_1, A_2 \in C$. If A_1 and A_2 are unifiable with mgu θ , add the clause $C[\theta]$.

$$\{D(f(y), y), \neg P(x), D(x, y), \neg P(z)\}$$

|

$$\{D(f(y), y), \neg P(f(y)), \neg P(z)\}$$

|

$$\{D(f(y), y), \neg P(f(y))\}$$



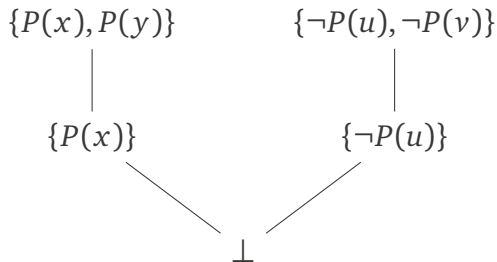
The Need for Factoring

Factoring is sometimes crucial:

$$\{P(x), P(y)\} \qquad \{\neg P(u), \neg P(v)\}$$

The Need for Factoring

Factoring is sometimes crucial:





The Resolution Method

Start with collection $\mathcal{C}$ of clauses

While $\perp \notin \mathcal{C}$ do

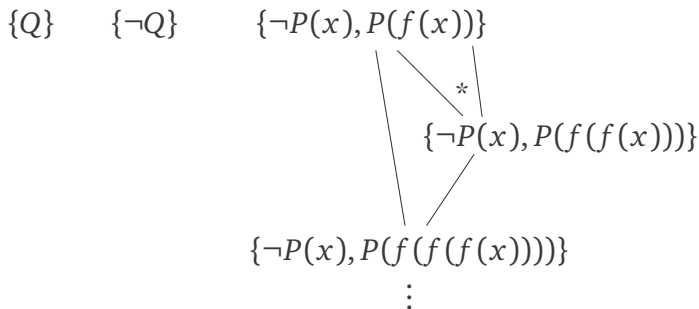
add to $\mathcal{C}$ a factor of some $C \in \mathcal{C}$
or a resolvent of some $C_1, C_2 \in \mathcal{C}$

Questions:

- Does this process always terminate for unsatisfiable inputs?
- For satisfiable inputs?

The Need for Fairness

For unsatisfiable inputs, only if we use a sensible search strategy. Consider:



* Here we resolve the clause against **a renamed copy of itself**. Without renaming, unification fails.



The Power of Resolution

Theorem. $\mathcal{C}$ is unsatisfiable iff the resolution method can add $\perp$ after a finite number of steps.

We say that resolution is **sound** and **refutation-complete**.

Not a **decision procedure**: may not terminate on satisfiable formulas.

Indeed, no such procedure can exist for predicate logic.

Validity/unsatisfiability are **semi-decidable** properties.